

Creacional

Estructural

Comportamiento

Fortnite

Spotify

PayPal

Netflix



Fortnite / Epic Games

Motor Unreal Engine — millones de jugadores simultáneos

Creacional

Factory Method

📌 Creación de armas, vehículos y objetos del mapa

Cada tipo de objeto del juego (pistola, escopeta, vehículo) tiene su propia fábrica. Agregar un nuevo ítem no modifica el código existente.

[ver pseudocódigo](#)

Comportamiento

Observer

📌 Sistema de eventos del juego (kills, zone shrink, eliminaciones)

El motor publica eventos (jugador eliminado, tormenta avanza) y múltiples sistemas se suscriben: HUD, sonido, logros, estadísticas.

[ver pseudocódigo](#)

Comportamiento

State

📌 Estados del personaje: corriendo, agachado, planeando, construyendo

El personaje delega su comportamiento al estado actual. Cambiar de estado cambia qué pueden hacer las acciones del jugador.

[ver pseudocódigo](#)

Creacional

Factory Method

📌 Creación de armas, vehículos y objetos del mapa

Cada tipo de objeto del juego (pistola, escopeta, vehículo) tiene su propia fábrica. Agregar un nuevo ítem no modifica el código existente.

[ocultar pseudocódigo](#)

```
// Interfaz base class Weapon { virtual shoot() = 0; } // Fábrica class WeaponFactory { static create(type: string): Weapon { if (type == "shotgun") return new Shotgun(); if (type == "sniper") return new SniperRifle(); } } // Al spawnear loot del cofre: Weapon w = WeaponFactory::create("shotgun");
```

Comportamiento

Observer

📌 Sistema de eventos del juego (kills, zone shrink, eliminaciones)

El motor publica eventos (jugador eliminado, tormenta avanza) y múltiples sistemas se suscriben: HUD, sonido, logros, estadísticas.

[ocultar pseudocódigo](#)

```
eventBus.on("player_eliminated", (data) => { HUD.updateKillFeed(data); // actualiza pantalla
AchievementsSystem.check(data); // revisa logros StatsTracker.record(data); // registra estadística
SoundSystem.playSound("elim"); // reproduce sonido }); // Al eliminar un jugador:
eventBus.emit("player_eliminated", { killer, victim });
```

Comportamiento State

📌 Estados del personaje: corriendo, agachado, planeando, construyendo

El personaje delega su comportamiento al estado actual. Cambiar de estado cambia qué pueden hacer las acciones del jugador.

[ocultar pseudocódigo ▸](#)

```
class Player { currentState: PlayerState; pressJump() { currentState.handleJump(this); } pressCrouch() {
  currentState.handleCrouch(this); } } // Cada estado implementa las acciones diferente: class GlidingState
implements PlayerState { handleJump(p) { /* no hace nada */ } handleCrouch(p) { p.setState(new
  FallingState()); } }
```



Spotify

Streaming de audio — 600M de usuarios activos

Estructural Decorator

📌 Pipeline de procesamiento de audio

El stream de audio pasa por decoradores encadenados: normalizador de volumen → ecualizador → crossfade → compresor. Cada uno agrega comportamiento sin modificar el anterior.

[ver pseudocódigo ▸](#)

Comportamiento Strategy

📌 Motor de recomendaciones musicales

El algoritmo de recomendación se intercambia según el contexto: "Discover Weekly" usa collaborative filtering, "Radio" usa content-based, "Daily Mix" usa una mezcla híbrida.

[ver pseudocódigo ▸](#)

Creacional Singleton

📌 Reproductor de audio del sistema

Solo puede existir una instancia del reproductor de audio activo en el dispositivo. El Singleton garantiza que no haya dos streams compitiendo.

[ver pseudocódigo ▸](#)



Spotify

Streaming de audio — 600M de usuarios activos

Estructural Decorator

📌 Pipeline de procesamiento de audio

El stream de audio pasa por decoradores encadenados: normalizador de volumen → ecualizador → crossfade → compresor. Cada uno agrega comportamiento sin modificar el anterior.

[ocultar pseudocódigo ▸](#)

```
let stream = new AudioStream(track); // Se apilan como capas: stream = new VolumeNormalizer(stream); stream
= new Equalizer(stream, userPrefs); stream = new CrossfadeDecorator(stream, 3000ms); // Al reproducir, cada
capa procesa y delega: stream.play();
```

Comportamiento Strategy

🔑 Motor de recomendaciones musicales

El algoritmo de recomendación se intercambia según el contexto: "Discover Weekly" usa collaborative filtering, "Radio" usa content-based, "Daily Mix" usa una mezcla híbrida.

[ocultar pseudocódigo](#)

```
interface RecommendationStrategy { recommend(user: User): Track[]; } class DiscoverWeekly implements RecommendationStrategy { recommend(u) { return collaborativeFilter(u); } } class RadioStrategy implements RecommendationStrategy { recommend(u) { return contentBasedFilter(u); } } // El contexto elige la estrategia: recommender.setStrategy(new DiscoverWeekly()); recommender.recommend(currentUser);
```

Creacional Singleton

🔑 Reproductor de audio del sistema

Solo puede existir una instancia del reproductor de audio activo en el dispositivo. El Singleton garantiza que no haya dos streams compitiendo.

[ocultar pseudocódigo](#)

```
class AudioPlayer { private static instance: AudioPlayer; static getInstance(): AudioPlayer { if (!instance) instance = new AudioPlayer(); return instance; } } // Desde cualquier parte del código: AudioPlayer.getInstance().play(track);
```



PayPal / Billeteras digitales

Procesamiento de pagos — millones de transacciones diarias

Comportamiento Chain of Responsibility

🔑 Pipeline de validación de transacciones

Cada transacción pasa por una cadena de handlers: validación de fondos → detección de fraude → cumplimiento regulatorio → autorización bancaria. Si uno falla, rechaza la transacción.

[ver pseudocódigo](#)

Estructural Adapter

🔑 Integración con múltiples pasarelas de pago y bancos

Cada banco tiene su propia API incompatible. El Adapter traduce la interfaz interna de PayPal a la API de cada banco externo (Visa, Mastercard, Banco local).

[ver pseudocódigo](#)

Comportamiento Command

🔑 Transacciones con soporte de undo (reembolsos)

Cada operación financiera es un objeto Command que puede ejecutarse, almacenarse en historial y revertirse (refund/chargeback).

[ver pseudocódigo](#)



PayPal / Billeteras digitales

Procesamiento de pagos — millones de transacciones diarias

Comportamiento Chain of Responsibility

🔑 Pipeline de validación de transacciones

Cada transacción pasa por una cadena de handlers: validación de fondos → detección de fraude → cumplimiento regulatorio → autorización bancaria. Si uno falla, rechaza la transacción.

[ocultar pseudocódigo](#)

```
// Cada handler decide si pasa o rechaza: class FraudDetectionHandler extends Handler { handle(tx: Transaction) { if (isSuspicious(tx)) return reject(tx); return next.handle(tx); // pasa al siguiente } } // Cadena configurada al inicio: chain = new FundsValidator().setNext(new FraudDetector()).setNext(new ComplianceChecker()).setNext(new BankAuthorizer()); chain.handle(transaction);
```

Estructural Adapter

🔑 Integración con múltiples pasarelas de pago y bancos

Cada banco tiene su propia API incompatible. El Adapter traduce la interfaz interna de PayPal a la API de cada banco externo (Visa, Mastercard, Banco local).

[ocultar pseudocódigo](#)

```
interface PaymentGateway { charge(amount, currency, token): Result; } // Adaptador para Visa (API distinta): class VisaAdapter implements PaymentGateway { charge(amount, currency, token) { // Traduce al formato que espera Visa: return visaApi.processPayment({ amt: amount, curr: currency, cvt: token }); } }
```

Comportamiento Command

🔑 Transacciones con soporte de undo (reembolsos)

Cada operación financiera es un objeto Command que puede ejecutarse, almacenarse en historial y revertirse (refund/chargeback).

[ocultar pseudocódigo](#)

```
class TransferCommand implements Command { execute() { debit(sender, amount); credit(receiver, amount); history.push(this); } undo() { debit(receiver, amount); credit(sender, amount); } } // Al procesar reembolso: lastCommand.undo();
```



Netflix

Streaming de video — arquitectura de microservicios a escala global

Creacional Abstract Factory

📌 Generación de familias de encodings por dispositivo

Netflix genera versiones del video para TV, móvil y web. Cada "fábrica de dispositivo" produce el codec, resolución y bitrate correctos para esa familia.

[ver pseudocódigo](#)

Estructural Proxy

📌 CDN y caché de contenido (Open Connect)

El servidor de video local actúa como Proxy del servidor central. Si tiene el contenido en caché, lo sirve directamente. Si no, lo pide al origen y lo guarda para la próxima vez.

[ver pseudocódigo](#)

Comportamiento Template Method

📌 Pipeline de procesamiento de contenido nuevo

Cuando sube un contenido nuevo, siempre se sigue el mismo esqueleto: ingest → validate → encode → thumbnail → publish. Cada tipo de contenido (película, serie, short) sobrescribe solo los pasos que difieren.

[ver pseudocódigo](#)



Netflix

Streaming de video — arquitectura de microservicios a escala global

Creacional Abstract Factory

📌 Generación de familias de encodings por dispositivo

Netflix genera versiones del video para TV, móvil y web. Cada "fábrica de dispositivo" produce el codec, resolución y bitrate correctos para esa familia.

[ocultar pseudocódigo](#)

```
interface DeviceFactory { createCodec(): Codec; createBitrate(): Bitrate; } class SmartTVFactory implements DeviceFactory { createCodec() { return new HEVC4KCodec(); } createBitrate() { return new HighBitrate(); } } class MobileFactory implements DeviceFactory { createCodec() { return new H264Codec(); } createBitrate() { return new AdaptiveBitrate(); } }
```

Estructural Proxy

📌 CDN y caché de contenido (Open Connect)

El servidor de video local actúa como Proxy del servidor central. Si tiene el contenido en caché, lo sirve directamente. Si no, lo pide al origen y lo guarda para la próxima vez.

[ocultar pseudocódigo](#)

```
class CDNProxy implements VideoService { serve(videoId, quality) { if (cache.has(videoId, quality)) { return cache.get(videoId, quality); // sirve local } const video = originServer.fetch(videoId, quality); cache.store(videoId, quality, video); // guarda return video; } }
```

Comportamiento Template Method

🔗 Pipeline de procesamiento de contenido nuevo

Cuando sube un contenido nuevo, siempre se sigue el mismo esqueleto: ingest → validate → encode → thumbnail → publish. Cada tipo de contenido (película, serie, short) sobrescribe solo los pasos que difieren.

[ocultar pseudocódigo](#) ▸

```
abstract class ContentPipeline { // Esqueleto fijo: process() { this.ingest(); this.validate();  
this.encode(); // ← sobrescribible this.thumbnail(); // ← sobrescribible this.publish(); } abstract  
encode(): void; abstract thumbnail(): void; } class MoviePipeline extends ContentPipeline { encode() {  
encodeIn4K_HDR(); } thumbnail() { extractKeyFrames(10); } }
```

Para el trabajo práctico

Identifiquen al menos 3 patrones en su trabajo integrador y justifiquen en qué componente estarían y por qué ese patrón y no otro.

Probar su implementación, esto se incorpora a la segunda entrega de trabajo.

Fortnite — por qué les importa el código del juego

El **Observer** es especialmente poderoso para discutir: cuando un jugador es eliminado, ¿cuántos sistemas necesitan saberlo? Kill feed, logros, ranking, sonido, estadísticas. Sin Observer, cada sistema estaría acoplado directamente a la lógica de combate. La pregunta para la clase: *¿qué pasa si Epic quiere agregar un nuevo sistema de torneos? ¿Cuánto código hay que tocar?*

Spotify — el Decorator es contraintuitivo y vale la pena

El pipeline de audio es el mejor ejemplo de Decorator porque los alumnos pueden visualizarlo físicamente: el sonido "pasa por" capas. Una buena pregunta socrática: *¿qué diferencia hay entre esto y simplemente llamar funciones en secuencia?* (La respuesta: las capas se pueden configurar en runtime, combinarse dinámicamente y el objeto de origen nunca cambia.)

PayPal — Chain of Responsibility como seguridad real

Este ejemplo conecta con seguridad informática. Pedir a los alumnos que piensen qué ocurre si el banco detecta fraude después de mover los fondos. Ahí entra el patrón **Command** con su `undo()`, que modela exactamente cómo funciona un chargeback.

Netflix — Abstract Factory + escala global

Netflix procesa miles de títulos nuevos por semana. El patrón **Template Method** en el pipeline de encoding es perfecto para discutir el principio *Open/Closed*: el esqueleto no cambia, pero agregar soporte para un nuevo tipo de contenido (por ejemplo, videos interactivos como Bandersnatch) solo requiere una nueva subclase.